Easypaisa

Submitted to: Dr. Nadir Shah

Submitted By:

Name	REG NO
Ayesha Noor	(FA25-BCS-022)
Alryan Sajjad	(FA25-BCS-052)
Ayesha Saddiqua	(FA25-BCS-170)

Date: 11-05-2026

Dept Of Computer Science

COMSATS University Islamabad, Wah campus

TABLE OF CONTENT

ASSIGNMENT : 04

1. Introduction

EasyPaisa is a JavaFX-based desktop application that simulates the core functionality of Pakistan's popular mobile wallet service. Built as an assignment in Object-Oriented Programming (OOP), it demonstrates key Java concepts including class design, encapsulation, ArrayList manipulation, regex-based input validation, and file-based data persistence.

The application provides a complete user lifecycle: account registration, secure PIN-based login, fund transfers, utility bill payments, money deposits, and a full transaction history. All data is stored in a plain-text file and reloaded on every application launch, ensuring state persists across sessions

2. Project Overview

2.1 Problem Statement

Mobile financial services like EasyPaisa are central to Pakistan's digital economy, yet their internal architecture is rarely explored in academic settings. This project aims to bridge that gap by rebuilding the core features of EasyPaisa as a standalone desktop application, giving team members hands-on experience with GUI programming, data validation, OOP design, and persistence.

2.2 Objectives

- Design and implement a multi-screen JavaFX application with consistent UI styling
- Apply OOP principles: classes, encapsulation, and instance methods
- Implement strict input validation using Java regular expressions
- Persist application data across sessions using Java File I/O (java.io)
- Simulate real financial operations: transfers, bill payments, and deposits
- Provide a clean, user-friendly interface modelled on the real EasyPaisa app

2.3 Scope & Constraints

The application is intentionally scoped to desktop use and does not include network connectivity, database integration, or cryptographic security. It is designed purely for

academic demonstration of Java and OOP concepts. All financial data is simulated; no real transactions occur.

3. Algorithm

The following steps describe the core algorithm of the EasyPaisa application:

Step 1. Application starts and loads saved data (users and transaction logs) from file.

Step 2. Login screen is shown. User enters mobile number and 5-digit PIN.

Step 3. Credentials are verified. If incorrect, an error is shown and the user tries again.

Step 4. On success, the Dashboard is displayed with the user's current balance and four action tiles.

Step 5. Send Money: receiver's phone is looked up, amount is validated, balance is transferred between accounts.

Step 6. Bill Payment: utility type is selected, amount is deducted from balance and logged.

Step 7. Add Money: deposit amount is added to the user's balance and recorded.

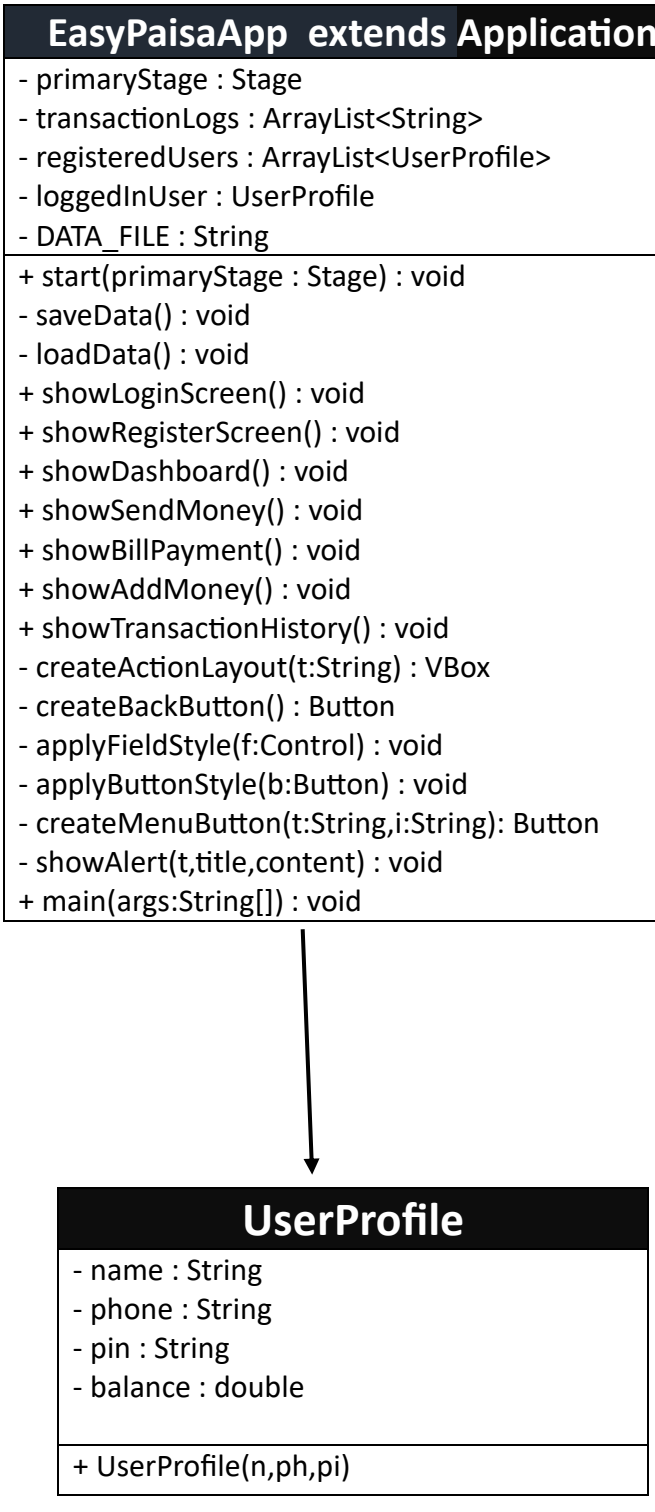
Step 8. History: all transactions are shown in reverse order in a list view.

Step 9. After every operation, updated data is saved to file to ensure persistence.

Step 10. Logout saves data and returns to the Login screen. Application ends when window is closed.

4. UML Class Diagram

The following UML class diagram illustrates the object-oriented structure of the EasyPaisa application, showing the two classes (EasyPaisaApp and UserProfile), their fields, methods, and the relationships between them:



5. Application Screens

The application comprises six screens, each encapsulated in its own method. The following sub-sections describe the UI layout, user interactions, and validation logic for each screen.

5.1 Login Screen (showLoginScreen)

The login screen is the entry point of the application. It features the EasyPaisa brand header (green StackPane with white bold logo text), two input fields, a login button, and a hyperlink to the registration screen.

Element	Detail
Mobile Number field	TextField; prompts "Mobile Number"; trimmed before validation
PIN field	PasswordField (masked); prompts "5-Digit PIN"; trimmed before validation
Login Button	Calls applyButtonStyle(); triggers validation lambda on click
Validation — empty check	If either field is empty -> ERROR alert: "Fields cannot be empty."
Validation — PIN format	If PIN does not match \d+ -> ERROR alert: "PIN must contain only numbers."
Create New Account	Hyperlink navigates to showRegisterScreen()

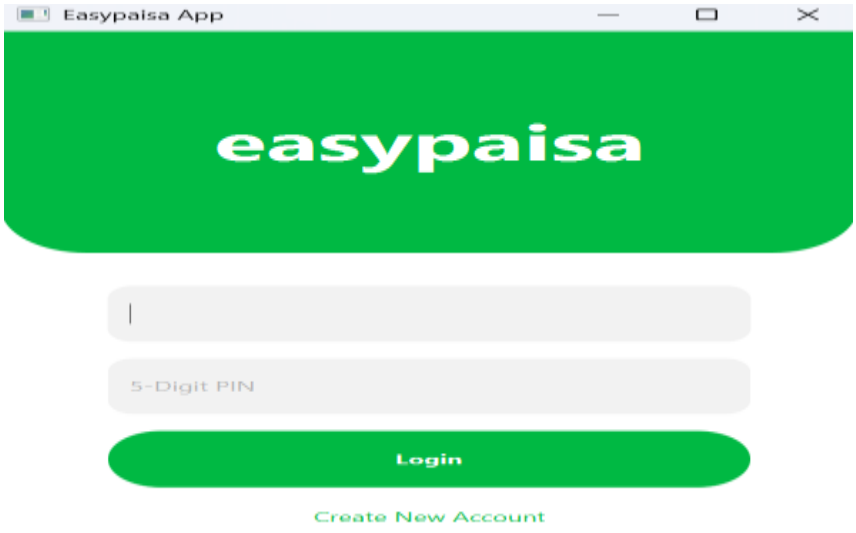


Figure 1: Login screen

5.2 Registration Screen (showRegisterScreen)

The registration screen collects Name, Phone, and PIN, validates each field independently with clear error messaging, and persists the new user immediately on success.

Field	Error Message on Failure
Full Name	Name is not written or invalid (use letters only)
Mobile Number	Mobile number is not written or invalid (use digits only)
PIN (presence)	PIN is not written
PIN (format)	PIN must contain only numbers. Letters are not allowed
PIN (length)	PIN size is not correct. It must be exactly 5 digits

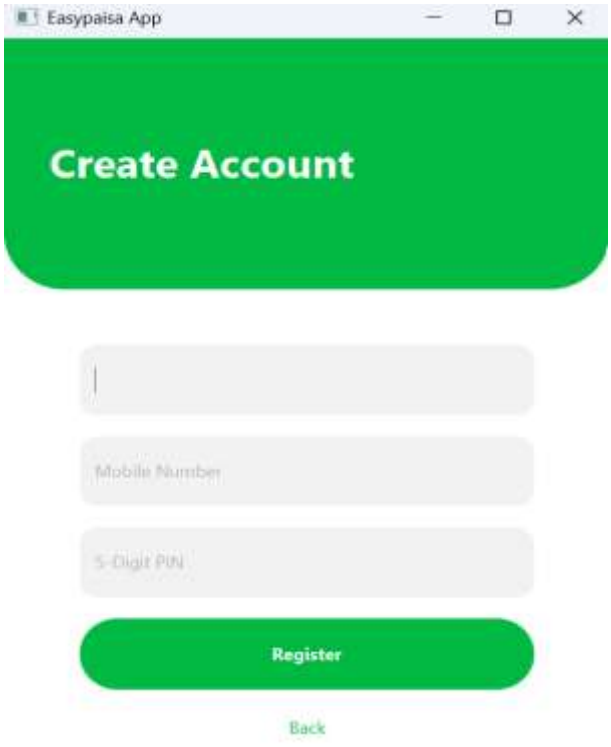


Figure 2: create new account

On successful validation, a new UserProfile is added to registeredUsers, saveData() is called immediately, an INFORMATION alert confirms success, and the app navigates back to the login screen.

5.3 Dashboard (showDashboard)

The dashboard is the central navigation hub. A green rounded header displays the current balance formatted as “Rs. XX.XX”. Below it, aGridPane hosts four feature tile buttons, each created by the createMenuButton() helper with an emoji icon and label.

- Send Money navigates to showSendMoney()
- Bill Payment navigates to showBillPayment()
- Add Money navigates to showAddMoney()
- History navigates to showTransactionHistory()
- Logout Button calls saveData() then showLoginScreen()

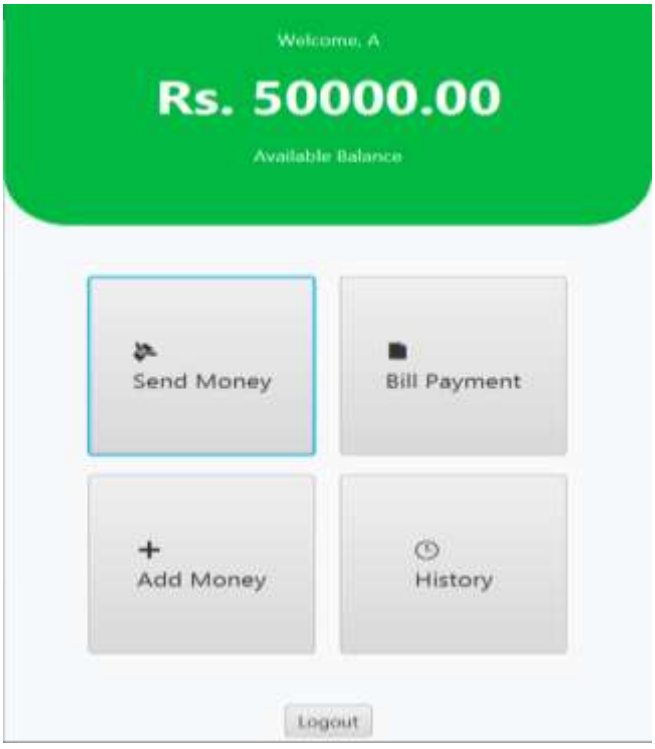


Figure 3: Dashboard

5.4 Send Money (showSendMoney)

Allows the logged-in user to transfer funds to another registered account.

- Recipient phone number is validated against the registeredUsers list via stream().anyMatch()

- Amount is parsed with Double.parseDouble(); a NumberFormatException triggers an ERROR alert
- Transfer is blocked if amount exceeds currentBalance (WARNING: “Insufficient funds.”)
- On success: currentBalance is decremented, a log entry is appended (“Sent to X: -Rs. Y”), saveData() is called, and the dashboard is shown

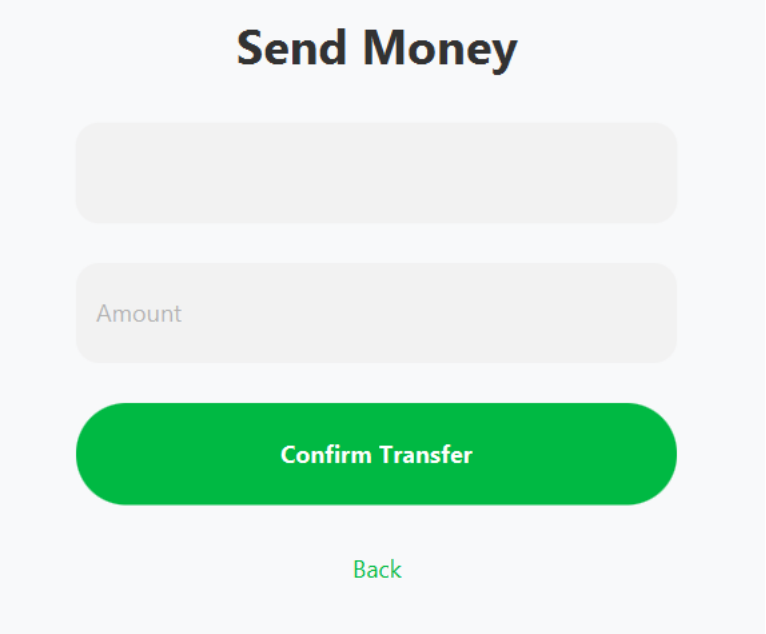


Figure 4: Sent money

5.5 Bill Payment (showBillPayment)

Handles utility bill payments across three provider types.

- Provider type selected via ComboBox<String> with options: Electric, Gas, Water
- Reference number TextField provided (stored in UI; not validated or persisted in this version)
- Amount validated via Double.parseDouble(); only proceeds if type is selected and val <= currentBalance
- Transaction logged as “[Type] Bill: -Rs. X”; balance updated and data saved

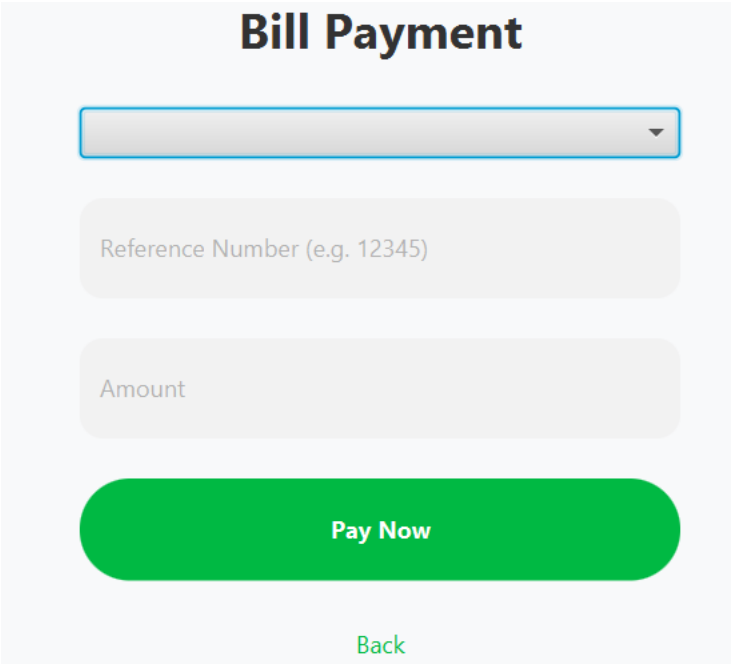


Figure 5: Bill Payment

5.6 Add Money (showAddMoney)

- Single TextField for deposit amount
- Parsed via Double.parseDouble(); only proceeds if val > 0
- Balance incremented; entry appended as “Deposit: +Rs. X”; data saved

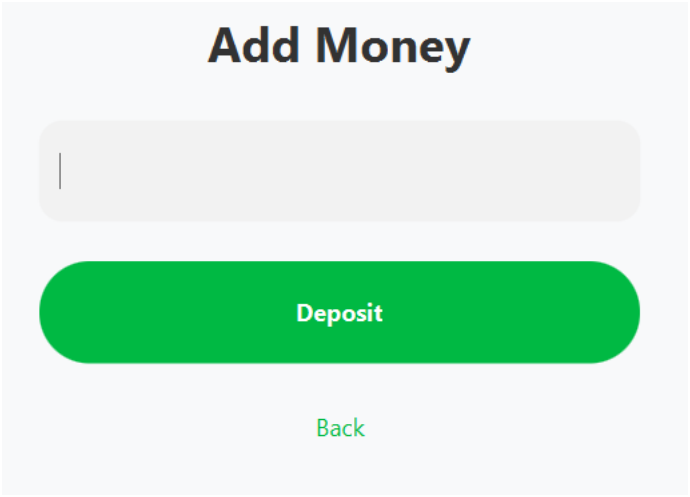


Figure 6: Add money

5.7 Transaction History (showTransactionHistory)

- ListView<String> populated by iterating transactionLogs in reverse (newest first)
- Shows all deposits, sends, and bill payments since account creation
- Read-only view; no filtering or search in current version

Figure 7: Transcation history

6. OOP Concepts Applied

This project demonstrates the following core Object-Oriented Programming principles:

OOP Principle	How It Is Applied in This Project
Classes & Objects	EasyPaisaApp and UserProfile are two distinct classes. EasyPaisaApp is instantiated by the JavaFX runtime via launch(); UserProfile objects are created for every new account.
Encapsulation	UserProfile bundles name, phone, and pin into a single object. EasyPaisaApp encapsulates all application state (balance, logs, users) and exposes behaviour only through public/private methods.
Inheritance	EasyPaisaApp extends javafx.application.Application, inheriting the JavaFX lifecycle and overriding the abstract start(Stage) method as the application entry point.
Abstraction	UI helper methods (applyFieldStyle, applyButtonStyle, createMenuButton, createBackButton, createActionLayout, showAlert) abstract repetitive styling and alert logic, hiding implementation details from screen methods.
Method Decomposition	Each screen is a self-contained method (show*Screen / show*). Data operations are separated into saveData() and loadData(). This single-responsibility decomposition improves readability and maintainability.
Collections & Generics	ArrayList<UserProfile> and ArrayList<String> use Java generics for type-safe storage. The Stream API (stream().anyMatch()) is used for declarative searching without explicit loops.

7. GUI Design & Styling

7.1 Design Language

The application follows the EasyPaisa brand palette: #00B943 (vibrant green) as the primary colour, white as the background, and #F8F9FA as the content area background. All interactive elements use the brand green to maintain visual consistency.

7.2 Reusable Helper Methods

Four helper methods centralise all styling decisions, ensuring every screen maintains a uniform look without duplicating CSS strings:

Method	What It Does
applyFieldStyle(Control f)	Sets height to 50px, max width to 300px, rounded grey background (#F2F2F2), border-radius 12, padding 10
applyButtonStyle(Button b)	Sets width/height to 300x50, white text, green background, border-radius 25, bold font
createMenuButton(label, icon)	Creates 140x140 white rounded tile with drop-shadow for dashboard grid
createBackButton()	Creates transparent button with green text; wires to showDashboard() by default
createActionLayout(title)	Returns VBox with #F8F9FA background, 30px padding, centred alignment, and title Label
showAlert(type, title, msg)	Instantiates JavaFX Alert with no header text; blocks until dismissed

7.3 Layout Hierarchy

Each screen uses JavaFX layout containers to organise content:

- Login / Register: VBox (root) -> StackPane/VBox (header) + VBox (form)
- Dashboard: VBox (root) -> VBox (header) + GridPane (2x2 tiles) + Button (logout)
- Feature screens: VBox returned by createActionLayout() with controls and back button added dynamically

8. Team Contributions

The project was divided equally among three group members. Each member took primary ownership of specific modules while collaborating on integration and testing.

Member	Responsibilities
Ayesha noor	showLoginScreen(), showRegisterScreen(), all PIN/phone validation logic, UserProfile class, saveData(), loadData()
Ayesha Saddiqua	showDashboard(), showSendMoney(), balance deduction logic, transaction log management, Send Money validation
Alryan sajjad	showBillPayment(), showAddMoney(), showTransactionHistory(), all helper methods (applyFieldStyle, applyButtonStyle, etc.), final integration, testing

9. Conclusion

This project successfully implements a functional digital wallet application that replicates the core experience of EasyPaisa using Java and JavaFX. It demonstrates practical application of OOP principles (encapsulation, inheritance, abstraction, and method decomposition) along side real-world concerns such as input validation, user authentication, and data persistence.

The team gained hands-on experience with JavaFX UI development, file-based persistence, and the Java Collections Framework. While the application has limitations appropriate to an academic project, the architecture is extensible, and the code is well-structured for future enhancements.